Assignment 2

Part 1

1) Phone_number=NULL will always give 0 because NULL means an unknown value, and by using = sign it results to unknown.
   The correct syntax is:
   SELECT COUNT(*)
   FROM student
   WHERE phone_number IS NULL
   NULL is excluded from comparisons and aggregate functions as it signifies the unknown data. The aggregate functions ignore NULL values to avoid treating unknown data as values.
2) Yes the student is correct. there is no use for the DISTINCT function here. The function GROUP BY department produces one row per department , so there is no repeatation of the departments.
3) If the LEFT JOIN returns more rows than INNER JOIN means that some customers did not place any order and they have the NULL written in the orders column. The INNER JOIN returns values for all the customers who have ordered. The LEFT JOIN shows all the customers who have ordered and also those who have not.
4) The problem in this query is with GROUP BY department, sql collapses all the rows of the same department into a single row. It means that if more than one person has the same department then they get squashed into one row

per department , this breaks the single row structure. Sql refuses to run it

The correct version is :

```
SELECT department, employee_name, AVG (salary)
FROM employee
GROUP BY department,employee_name;
```

5) The correct query:

```
SELECT department , AVG(salary) AS avg_salary
FROM employee
GROUP BY department
HAVING AVG(salary)>80000;
```

WHERE runs before grouping, so AVG(salary) hasn't been calculated yet. HAVING runsafter grouping so the average is ready to be filtered on.

6) A) Non correlated subquery .

```
SELECT product_name,price
FROM products
WHERE price>(SELECT AVG(price) FROM products);
```

The average value is a single fixed value that is independent of any row in the query, the subquery can be computed once and reused for every row comparison .

b) correlated subquery .

```
SELECT*
FROM employee as e
WHERE salary > ( SELECT AVG(salary)
                 FROM employees as es
                  WHERE
es.department_id=e.department_id);
```

The avg salary depends on which department the current outer row belongs to. The inner query must refer

e.department_id from the outer query,so it cant be evaluated independently.

Part 2

a) SELECT c.customer_id, c.name, c.email, c.phone
FROM customer c
LEFT JOIN ORDER ON c.customer_id=o.customer_id
WHERE o.order_id IS NULL;
Explanation: inner join only keeps customers who have same orders , left join saves all the customers even if there are no orders for them,so WHERE o.order_id IS NULL separates the never ordered.

b) SELECT
r.restaurant_id,r.name,AVG(order_total.order_value) AS avg_order_value
FROM restaurant r
JOIN (SELECT o.order_id,o.restaurant_id,
SUM(mi.price*oi.quantity) AS order_value
FROM order o
JOIN orderitem oi on o.order_id=oi.order_id
JOIN menuitem mi ON oi_item_id=mi.item_id
GROUP BY o.order_id,o.restaurant_id) AS order_totals
ON r.restaurant_id=order_totals.restaurant_id
GROUP BY r.restaurant_id,r.name
HAVING COUNT(order_totals.order_id)>5;
Explanation: Order value is not a stored column — it's a SUM over joined orderItem/menuItem rows — so a derived table first computes per-order totals, then an outer aggregation averages those per restaurant.

c) WITH restaurant_avg AS (

```
SELECT r.restaurant_id, r.name,
AVG(order_totals.order_value) AS avg_order_value

FROM Restaurant r

JOIN (

SELECT o.order_id, o.restaurant_id, SUM(mi.price *
oi.quantity) AS order_value

 FROM order o

 JOIN orderItem oi ON o.order_id = oi.order_id

JOIN menuItem mi ON oi.item_id = mi.item_id

GROUP BY o.order_id, o.restaurant_id ) AS order_totals ON
r.restaurant_id = order_totals.restaurant_id

GROUP BY r.restaurant_id, r.name

HAVING COUNT(order_totals.order_id) > 5 )

SELECT * FROM restaurant_avg

 WHERE avg_order_value > (SELECT AVG(avg_order_value)
FROM restaurant_avg);
```

Explanation: The syntax first computes order values, then restaurant-level averages, and finally compares them to the platform-wide average order value. A correlated subquery is not needed because the platform-wide average is a single numerical value that does not depend of the outer restaurant row.

d)
```
WITH item_totals AS
    ( SELECT mi.restaurant_id, mi.item_id, mi.item_name,
    SUM(oi.quantity) AS total_quantity
```

```
FROM MenuItem mi
JOIN orderItem oi ON mi.item_id = oi.item_id
GROUP BY mi.restaurant_id, mi.item_id, mi.item_name ),
max_per_restaurant AS ( SELECT restaurant_id,
MAX(total_quantity) AS max_quantity FROM item_totals
GROUP BY restaurant_id )
 SELECT it.restaurant_id, it.item_name, it.total_quantity
FROM item_totals it
JOIN max_per_restaurant m
ON it.restaurant_id = m.restaurant_id
 AND it.total_quantity = m.max_quantity;
```

Explanation: sql will not let the GROUP BY restaurant_id query also return item_name directly with MAX(total_quantity),since item_name is not dependent on the grouping column we compute per item totals and find the max per restaurant separately then join back to recover the item name.

Part 3

Q1) student_id--student_name,
course_id--course_name,instructor_id,
Instructor_id--instructor_name,instructor_office,
(student_id,course_id)--grade,enrollment_date,
Course_id--instructor_name,instructor_office.

Q2) yes, the table is in 1NF. A relation is in 1NF if each attribute has single values,there are no multivalued attributes, and each row is uniquely differentiable. In the table given:
Student_name,course_name,instructor_name,instructor_office,grade,enrollment_date all contain single values.

No attribute has multiple values.
The composite key I.e. (student_id,course_id) uniquely identifies each enrollment record.

Q3) A realtion in 2NF must be in 1NF and should not have any partial dependencies on the composite key.
Dependencies that violate are:
1) student_id--student_name: student_name only depends on student_id of the key.
2) Course_id--course_name,instructor_id: this also depends only on course_id of the key.

Q4) a relation in 3NF must have no transitive dependencies.hence:
course_id--instructor_id
Instructor_id--instructor_name,instructor_office.

Course_id--instructor_id--instructor_name,instructor_office.

Therefor, the transitive dependency is:

Course_id--instructor_name,instructor_office.

Anomaly:
Update anomaly: if an instructor shifts from one office to another, because instructor_office I stored in every enrollment roe for course taught by that instructor many rows must be updated, if some rows are updated and some are not then it creates an anomaly.

Deletion anomaly: if an enrollement in a course is deleted the information about the instructor will be gone even though the instructor exists.

Q5) A) student:
Student(student_id,student_name)
Primary key: student_id
Functional dependency: student_id--student_name
Reason: student_name only depends on student_id

B) Instructor
Instructor(instructor_id,instructor_name,instructor_office)
Primary key: instructor_id
Functional dependency: instructor_id--
instructor_name,instructor_office
Reason: instructor details depend only on instructor_id.

C) Course
Course(course_id,course_name,instructor_id)
Primary key: course_id
Functional dependency: course_id--
course_name,instructor_id
Reason: a course has only one instructor and one name.

D) Enrollment
Enrollment(student_id,course_id,grade,enrollment_date)
Primary key: (student_id,course_id)
Functional dependency: (student_id,course_id)--
grade,enrollemnt_date.

Part 4

A) From c) i may have used a subquery to compare each restaurants avg order values with platform wise avg order value. I chose subquery because platform wide avg is a single aggregate value that can be calculated once and then compared with other restaurants average . this way Is easier to understand and clearly separates the two levels of aggregation. A JOIN based solution would require creating derived tables and then joining them together , which makes it more difficult to understand. If using a non correlated subquery avoids repeated execution for each row which provides better performance than a correlated approch.

B) Normalizing the enrollment table into 3NF reduces redundancy and prevents update, insertion and deletion anomaly, but it can make some queries less convenient. Example, retrieving a students enrollment information together with course and instructor details now requires joining multiple tables instead of reading from a single one. These joins can increase query complexity and execution cost.